

Blackthorn

Security Review For Morpho

Collaborative Audit Prepared For: **Morpho**

Lead Security Expert(s): **0x73696d616f**

hyh

xiaoming90

Date Audited: **August 18 - August 19, 2026**

Introduction

BlueFallbackRolling permissionlessly refinances a borrower's Midnight debt into a user-selected Morpho Blue market once the Midnight market reaches the configured start timestamp. It supplies the activated collateral to Blue before, through a callback, borrowing against it on Blue to reward the caller, repay the debt and withdraw the collateral on Midnight.

Users and accounts they authorize on Midnight may enable multiple fallback configurations per Midnight market. Users must authorize the contract on both Midnight and Blue. Each configuration selects a Blue market, a Midnight market, start timestamp, end timestamp, caller incentive at start, caller incentive at end, and minimum rollable amount, and can later be disabled. The caller incentive is a percentage of the debt, is capped at 100% and increases linearly between start and end. The minimum rollable amount is the smallest debt a single roll can migrate, and is bypassed by a roll migrating the whole remaining Midnight debt, so a position can always be closed out. A roll requires the borrower to have exactly one activated Midnight collateral, matching the collateral token of the configured Blue market.

Scope

Repository: [morpho-org/midnight](https://github.com/morpho-org/midnight)

Audited Commit: [3df4c22647a206a1d05f8784326d814a5eb6f50b](#)

Final Commit: [12503f275589d273bc39eb4b5a8460b8bbbed4c18](#)

Files:

- [src/periphery/blue-fallback-rolling/BlueFallbackRolling.sol](#)

Final Commit Hash

[12503f275589d273bc39eb4b5a8460b8bbbed4c18](#)

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	0	5

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Security Experts Dedicated to This Review

@0x73696d6166

A handwritten signature in white ink on a black background. The signature consists of a large, stylized capital 'S' that loops around the word 'Simao', which is written in a cursive script.

@hyh

A handwritten signature in white ink on a black background. The signature is a stylized, cursive 'hyh'.

@xiaoming90

A handwritten signature in white ink on a black background. The signature is a stylized, cursive 'Xiaoming90'.

Issue L-1: BlueFallbackRolling::roll() reverts with a division-by-zero panic when the user has no debt [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit2/2026-08-morpho-blue-fallback-rolling-aug-18th-2026/issues/17>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

BlueFallbackRolling::roll() divides by IMidnight::debt() without checking it is non null, so a roll on a position with no debt reverts with panic 0x12.

Vulnerability Detail

BlueFallbackRolling::roll() reads the user's debt from IMidnight::debt() and uses it as the denominator when prorating the collateral to move.

It uses UtilsLib::mulDivDown() call on the IMidnight::collateral() amount, which performs $(x * y) / d$ in plain Solidity with $d == 0$ and panics.

This is reached whenever a roller calls BlueFallbackRolling::roll() on a configured position whose debt has already been repaid or rolled in full, which is the normal end state of the configuration.

Impact

It may be hard to find out why the transaction failed, and should be common as bots will rush to roll the position to get the incentive.

Code Snippet

https://github.com/sherlock-audit2/2026-08-morpho-blue-fallback-rolling-aug-18th-2026/blob/main/morpho-org__midnight/src/periphery/blue-fallback-rolling/BlueFallback

Rolling.sol#L94-L100

Tool Used

Manual Review

Recommendation

Require `debtAssets` to be non null in `BlueFallbackRolling::roll()`, with a dedicated error added to `IBlueFallbackRolling`.

Discussion

peyha

We'll acknowledge this one. We usually don't make errors more explicit as per our guidelines

Issue L-2: BlueFallbackRolling::roll() pays the incentive at the inclusion timestamp with no minimum, so a delayed roll settles below what the roller priced [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-08-morpho-blue-fallback-rolling-aug-18th-2026/issues/18>

Summary

`BlueFallbackRolling::roll()` derives the roller's incentive from `block.timestamp` at inclusion and takes no minimum from the caller. On a decaying schedule the roller is paid whatever the incentive has decayed to by the time the transaction lands, which is strictly less than the amount they priced when they sent it.

Vulnerability Detail

`BlueFallbackRolling::setConfig()` accepts `incentiveAtEnd` below `incentiveAtStart`, and `BlueFallbackRolling::roll()` then interpolates downwards with `UtilsLib::mulDivUp()` over `block.timestamp - start`.

A roller who computes the incentive off-chain and sends the transaction is paid at whatever timestamp the transaction is eventually mined at, so any gap between signing and inclusion (mempool congestion, a sequencer outage, a chain halt, a reorg that drops and re-includes the transaction) will decrease their payment.

Impact

The roller performs the roll for less than the incentive they priced, down to `incentiveAtEnd` and possibly zero, while still paying the gas.

Code Snippet

https://github.com/sherlock-audit2/2026-08-morpho-blue-fallback-rolling-aug-18th-2026/blob/main/morpho-org__midnight/src/periphery/blue-fallback-rolling/BlueFallbackRolling.sol#L102-L113

```
// Round against the roller.
uint256 incentiveFactor = incentiveAtEnd >= incentiveAtStart
    ? incentiveAtStart
      + UtilsLib.mulDivDown(incentiveAtEnd - incentiveAtStart, block.timestamp -
        ↪ start, end - start)
    : incentiveAtStart
      - UtilsLib.mulDivUp(incentiveAtStart - incentiveAtEnd, block.timestamp -
        ↪ start, end - start);
uint256 incentiveAssets = UtilsLib.mulDivDown(assets, incentiveFactor, WAD);
```

Tool Used

Manual Review

Recommendation

Add a `minIncentiveAssets` parameter to `BlueFallbackRolling::roll()` and require `incentiveAssets >= minIncentiveAssets`, so the roller sets their own floor instead of accepting the inclusion timestamp's price.

Discussion

peyha

indirectly fixed by [#1155](#)

Issue L-3: Rounding the rolled collateral down does not preserve the Midnight health check, DoSing BlueFallbackRolling::roll() at the health boundary [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-08-morpho-blue-fallback-rolling-aug-18th-2026/issues/19>

Summary

BlueFallbackRolling::roll() claims that rounding the withdrawn collateral down rounds in favor of the Midnight position. That is not true of Midnight::isHealthy(), which floors the collateral side twice, once on the oracle price and once on the LLTV. A proportional roll can therefore lower the Midnight health ratio instead of leaving it unchanged.

Vulnerability Detail

BlueFallbackRolling::roll() takes the rolled collateral as a share of the position proportional to the rolled debt, rounded down with UtilsLib::mulDivDown() over Midnight::collateral() and Midnight::debt(). BlueFallbackRolling::onMorphoSupplyCollateral() then repays assets through Midnight::repay(), which subtracts exactly assets from the position debt, and withdraws collateralAssets through Midnight::withdrawCollateral(), which requires the resulting position to be healthy.

The reasoning behind the comment holds only for an exact, unrounded borrowing power. Midnight::isHealthy() computes it as $\text{floor}(\text{floor}(\text{collateral} * \text{price} / \text{ORACLE_PRICE_SCALE}) * \text{lltv} / \text{WAD})$, so it is not linear in the collateral: scaling the collateral down by the same factor as the debt can cost up to two extra units of borrowing power that the debt reduction does not pay for. Keeping a wei more collateral on Midnight does not compensate for those two floors.

Hence, it's possible the health check in Midnight::withdrawCollateral() fails and the whole roll reverts.

Impact

A Midnight position sitting within about two units of loan token of its `Midnight::isHealthy()` limit, which is where a borrower who borrowed the maximum lands, has `BlueFallbackRolling::roll()` DoSed for every partial amount, exactly the borrower the fallback exists for.

Code Snippet

BlueFallbackRolling::roll()

```
// Round in favor of the Midnight position. Thus, splitting the rolls can raise the
↳ blue final LTV. This is
// mitigated by the min rollable debt.
uint256 collateralAssets =
    IMidnight(MIDNIGHT).collateral(midnightId, user,
↳ collateralIndex).mulDivDown(assets, debtAssets);
```

Midnight::isHealthy()

```
maxDebt += _position.collateral[i].mulDivDown(price, ORACLE_PRICE_SCALE)
    .mulDivDown(collateralParam.lltv, WAD);
```

Tool Used

Manual Review

Recommendation

Correct the comment: the rounding favors the Midnight position in collateral terms, not in `Midnight::isHealthy()` terms. To make a partial roll always leave the Midnight position at least as healthy, reduce the withdrawn collateral by a buffer.

Discussion

peyha

The comment is correct for me because it refers to the use of `mulDivDown` instead of `mulDivUp` but we could precise that the health of midnight is not necessarily improving

peyha

We refined the comment in this PR. We ended up not mentioning that `debt / maxDebt` can increase because

1. it's quite subtle and introduces the concept of health which is ambiguous
2. similar cases exist on midnight and have not been documented

Issue L-4: Roll ignores Midnight's liquidation lock letting takers overfill borrowers past Midnight's LLTV [RESOLVED]

Source: <https://github.com/sherlock-audit2/2026-08-morpho-blue-fallback-rolling-aug-18th-2026/issues/20>

Vulnerability Detail

`Midnight.take` raises a seller's debt before validating it, and defers the health check until after its callbacks. As a result, the seller debt is inflated during the callback windows.

A transient liquidation lock mechanism is implemented to mitigate the risk of malicious users who gain control of the execution during the callback from liquidating the seller with inflated debt during that windows.

```
File: Midnight.sol
363:     function take(
    ..SNIP..
443:         sellerPos.credit -= UtilsLib.toUint128(sellerCreditDecrease);
444:         sellerPos.debt += UtilsLib.toUint128(sellerDebtIncrease); // seller
    ↪ debt increased, NOT yet validated
    ..SNIP..
480:         bool wasLocked = UtilsLib.tExchange(LIQUIDATION_LOCK_SLOT, id, seller,
    ↪ true); // seller quarantined from liquidation
481:         if (buyerCallback != address(0)) {
482:             bytes memory buyerCallbackData = offer.buy ? offer.callbackData :
    ↪ takerCallbackData;
483:             require(
484:                 IBuyCallback(buyerCallback)
485:                     .onBuy(id, offer.market, buyerAssets, units,
    ↪ buyerPendingFeeIncrease, buyer, buyerCallbackData)
486:                     == CALLBACK_SUCCESS,
487:                 WrongBuyCallbackReturnValue()
488:             );
```

```

489:         }
490:
491:         SafeTransferLib.safeTransferFrom(offer.market.loanToken, payer,
↪ address(this), buyerAssets - sellerAssets);
492:         SafeTransferLib.safeTransferFrom(offer.market.loanToken, payer,
↪ receiver, sellerAssets);
..SNIP..
511:         if (!wasLocked) UtilsLib.tExchange(LIQUIDATION_LOCK_SLOT, id, seller,
↪ false); // lock cleared, deferred health check
512:         require(liquidationLocked(id, seller) || isHealthy(offer.market, id,
↪ seller), SellerIsLiquidatable());
513:
514:         return (buyerAssets, sellerAssets);
515:     }

```

For the `BlueFallbackRolling.roll` function, there is also a similar risk where the caller might leverage the inflated user's debt. This causes the borrower to borrow more than the normal route, silently changing the borrower's LTV ceiling from Midnight's LLTV to Blue's.

Consider the following scenario:

- Alice: 10 WETH (\$30,000) collateral, 20,000 USDC debt on Midnight (LLTV 0.86, so max debt 25,800).
- Alice has a live sell offer for up to 20,000 more units at 0.99507 (fee 2 bp), and an armed BFR config to a Blue WETH/USDC market with LLTV 0.945 (so max borrow 28,350), incentive 0.1%.
- Mallory: unrelated address, no capital, no authorizations.

Honest limit: Midnight lets anyone fill at most 5,800 units ($20,000 + 5,800 = 25,800$).

Malicious limit: Blue allows $(20,000 + u) \times 1.001 \leq 28,350 \Rightarrow u = 8,321.68$.

1. Mallory takes 8,321.68 units of Alice's offer, with her own contract as callback. Midnight immediately credits Mallory 8,321.68 credit units and sets Alice's debt to 28,321.68, above Midnight's cap, then calls Mallory's callback before collecting payment.
2. Inside the callback, Mallory rolls Alice's full 28,321.68. Blue lends 28,350.00 (debt +

0.1% incentive) against the 10 WETH; the health check passes with \$0.000001 to spare. BFR repays Midnight 28,321.68 (Alice's Midnight debt goes to 0 and withdrawable rises by 28,321.68), moves the 10 WETH to Blue, and pays Mallory the 28.32 incentive.

3. Still inside the callback, Mallory withdraws her 8,321.68 credit at par from the withdrawable the roll just created: +8,321.68 USDC.
4. Callback returns; Midnight collects 8,282.32 from Mallory (8,280.65 to Alice, 1.66 fee). The deferred health check on Alice sees debt = 0 and passes.

The following is the result before and after:

	before	after
Mallory	0 USDC	+67.68 USDC (39.36 par gain + 28.32 incentive), zero collateral
Alice	\$10,000 equity, Midnight LTV 66.7%	\$9,930.65 equity, Blue LTV 94.5000% against a 94.59% threshold

Honest path for comparison: fill capped at 5,800, roll lands at Blue LTV 86.09% with \$2,524 of margin.

Midnight is supposed to guarantee a maker cannot be filled past its own LLTV. Running a roll inside the take callback converts that bound into Blue's looser one (here 8.5 percentage points of collateral value), lands the borrower with zero margin, and costs the taker nothing.

Impact

Any taker can fill a borrower's sell offer beyond Midnight's LLTV by rolling the inflated debt inside the take callback. Midnight's deferred health check is bypassed, the borrower lands on Blue at its exact liquidation threshold with zero margin, and the attacker profits with no capital or authorization.

Code Snippet

https://github.com/sherlock-audit/2026-08-morpho-blue-fallback-rolling-aug-18th-2026/blob/0d992656a57908c5ff7e1c98c10c8fa79a7bd875/morpho-org__midnight/src/per

Tool Used

Manual Review

Recommendation

Consider adding `require(!IMidnight(MIDNIGHT).liquidationLocked(midnightId, user))` in `roll`, so that `roll` will be refused while the borrower's Midnight position is liquidation-locked and while debt is inflated.

Discussion

peyha

This would require the borrower to have an open borrow (sell) offer while the rolling config is active which is very unlikely (because the goal of the user is to close the borrow position). Still I think it opens a path that we don't want to allow (the roller manipulating the health of the resulting blue position) and the fix is straightforward so I'll implement it

peyha

That's a nice catch, we fixed it in [this PR](#) by implementing your recommendation

Issue L-5: Caveats: roller-lender path can be temporary blocked if not done atomically, roll can be permanently blocked when collateral move and incentives breach Blue' LLTV [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-08-morpho-blue-fallback-rolling-aug-18th-2026/issues/21>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

- 1) All three parts (sizing, moving, roll) of lender supply-roll use case need to be done atomically in order to remove the risk of front-running. Otherwise the roll incentive can be stolen or the call can be denied by front running attacker.
- 2) Roll parameters are fixed in advance by the borrower. Collateral value fluctuates and can make the given incentive plus Midnight to Blue LLTV transition impossible as a whole. A possible non-proportional move, that can be beneficial both to roller and borrower, is not allowed by design.

Vulnerability Detail

- 1) Alice the roller who in order to roll needs to provide the liquidity to the underlying illiquid Blue market first, for example by moving her Midnight position there, has to do it atomically, as otherwise the liquidity can be used by someone else and roll becomes impossible as stated in call parameters or the roll itself can be front run wholly, capturing the incentive without the funding. In addition, the natural $assets = debt$ setting that can be used by Alice to maximize the incentive can be grieved by a front runner rolling with `minRollableAssets`, so for Alice it becomes $assets = debt + minRollableAssets > debt$ and her call reverts. I.e. to be safe Alice must do

all three steps atomically: get current debt, provide the same amount of liquidity plus the incentive, roll.

- 2) If collateral value plunges closer to `start` timestamp the proportional move might become impossible due to incentive addition to new debt and Blue's LLTV compared to Midnight's. In the same time it might be possible to leave a not proportional part of debt on Midnight, i.e. increase the risk of a smaller part on Midnight and leave it there and roll the remaining biggest part to Blue to save it from liquidation, which is not an option by design. I.e. if a roller can't move proportionally and wants to do a possible non-proportional move for the incentive (which is potentially profitable both for them and borrower) they can't, and the whole borrower's position can be liquidated instead.

Impact

- 1) If lender supply-roll use case is done not atomically then it can be front run with incentive stealing (attacker not providing the lender part of funds might capture the incentive) and roll call blocking (attacker can deny the operation as a grieving).
- 2) Non-proportional move is not possible, while collateral plunge might leave no other options. Borrower's roll can be blocked, their position might be liquidated after maturity. I.e. functionality impact is that borrower can't setup a roll and go away, they have to monitor collateral value not only bad debt wise (which is always implied in lending), but incentive plus Midnight to Blue LLTV transition wise.

Code Snippet

- 1) If `assets > debtAssets` then the roll call will be denied:

BlueFallbackRolling.sol#L130

```
IMidnight(MIDNIGHT).repay(midnightMarket, assets, user, address(0), hex "");
```

- 2) Collateral to debt correspondence is fixed to be proportional:

BlueFallbackRolling.sol#L99-L100

```
uint256 collateralAssets =
```

```
IMidnight(MIDNIGHT).collateral(midnightId, user,  
    ↪ collateralIndex).mulDivDown(assets, debtAssets);
```

Tool Used

Manual Review

Recommendation

Consider documenting (1) and (2) as roller and borrower caveats.

Discussion

peyha

1. is a well known issue. Liquidators can also be frontrunners when trying to liquidate on midnight and it was raised during audits. Rollers can solve this by implementing a short smart contract which will adjust the debt to roll on the fly so we'll acknowledge the issue.
2. was also discussed during design. We chose not to consider potential health based liquidations because the main goal of rolling is to prevent post-maturity liquidation (which is not health based). Managing the health of positions (on Midnight and on Blue with the incentive) is a distinct problem which is not covered by the scope of this contract

Disclaimers

Blackthorn does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.